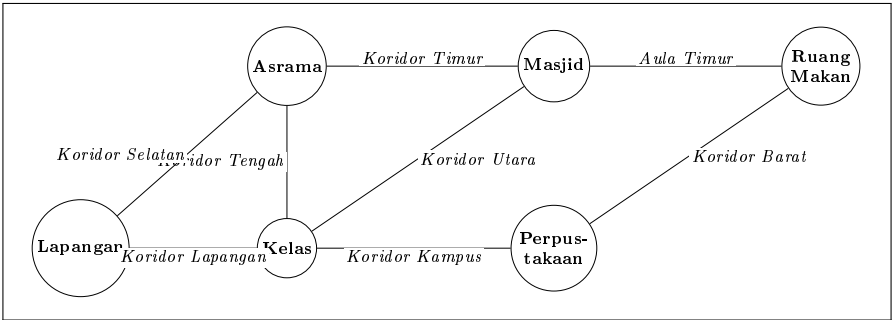


SPESIFIKASI TUGAS BESAR CAMPUSFLOW

Sistem Optimasi Arus Perpindahan Kelompok Mahasiswa Berbasis Simulasi dan Local Search

Mata Kuliah: **Inteligensi Artifisial**

Repository: Task2_AI_13524065



Nama	Kurt Mikhael Purba
NIM	13524065

Informasi Dokumen

Elemen	Keterangan
Judul sistem	CampusFlow: Optimasi Pergerakan Kelompok Mahasiswa pada Pergantian Jadwal Perkuliahan
Topik	Local Search dengan complete-state formulation
Algoritma	Basic, Sideways Move, Stochastic, dan Random-Restart Hill-Climbing, Simulated Annealing, Genetic Algorithm
Bahasa implementasi	Python 3 (tanpa pustaka eksternal)
Platform	Terminal (output berbasis teks)
Target implementasi	Spesifikasi lengkap sebagai acuan implementasi dan proof of concept fitur inti
Format repository	Task2_AI_13524065

Catatan penggunaan: Dokumen ini dirancang sebagai acuan implementasi. Nama kelompok, kapasitas koridor, dan angka keseimbangan boleh disesuaikan selama mekanisme inti, fungsi utama, konsep wajib, serta komponen objective function tetap terpenuhi dan perubahan didokumentasikan.

Daftar Isi Ringkas

1. Latar belakang dan tujuan tugas besar
2. Tujuan dan deskripsi permasalahan
3. Representasi state (complete-state formulation)
4. Data yang digunakan
5. Aturan serta batasan permasalahan (constraints)
6. Formulasi initial state
7. Successor function dan neighbor move
8. Objective function
9. Penerapan algoritma local search
10. Daftar fungsi dan command utama Python
11. Contoh input
12. Contoh output
13. Batas proof of concept, pengujian, dan deliverables
14. Penutup
15. Lampiran: kode sumber proof of concept

1. Latar Belakang dan Tujuan Tugas Besar

Topik. Membangun dokumen spesifikasi dan proof of concept Tugas Besar Inteligensi Artifisial dengan topik Local Search. Sistem yang dirancang, **CampusFlow**, mengoptimalkan pergerakan kelompok mahasiswa ketika terjadi pergantian jadwal perkuliahan. Permasalahan diformulasikan sebagai persoalan local search berbasis *state lengkap* (complete-state formulation): setiap state sudah memuat keputusan lengkap untuk seluruh kelompok, sehingga algoritma tidak perlu mencari path menuju goal.

Tujuan akademik. Menggabungkan materi Local Search, mulai dari konsep state, neighbor, dan objective function, hingga Hill-Climbing, Simulated Annealing, dan Genetic Algorithm dalam satu program yang dapat dijalankan, diuji, dan dikembangkan secara modular. Implementasi wajib memperlihatkan penerapan konsep yang diminta secara nyata, bukan sekadar fungsi yang tidak berhubungan dengan permasalahan.

- Memformulasikan permasalahan nyata sebagai persoalan local search dengan state lengkap.
- Merancang representasi state, successor/neighbor function, dan objective function multikriteria.
- Mengimplementasikan Basic (Steepest-Ascent), Sideways Move, Stochastic, dan Random-Restart Hill-Climbing, Simulated Annealing, serta Genetic Algorithm (populasi, selection, crossover, mutation).
- Menyediakan proof of concept yang memvisualisasikan state awal, state akhir, dan nilai objective function selama proses pencarian.
- Mendokumentasikan data, command utama, contoh input, contoh output, dan batasan implementasi.

2. Tujuan dan Deskripsi Permasalahan

2.1 Deskripsi Permasalahan

Pada pergantian jadwal perkuliahan, sejumlah kelompok mahasiswa harus berpindah tempat secara bersamaan. Sebagai contoh:

- Kelompok A bergerak dari Asrama menuju Masjid.
- Kelompok B bergerak dari Kelas menuju Masjid.
- Kelompok C bergerak dari Perpustakaan menuju Ruang Makan.
- Kelompok D bergerak dari Lapangan menuju Asrama.

Ketika beberapa kelompok menggunakan koridor, tangga, atau pintu yang sama dalam waktu yang bersamaan, dapat timbul kepadatan di koridor, antrean di pintu, tabrakan arus dari arah berlawanan, keterlambatan mahasiswa, hingga risiko keamanan ketika jumlah mahasiswa melebihi kapasitas jalur.

Sistem harus menentukan dua hal untuk setiap kelompok:

1. **Jalur (route)** yang digunakan; dan
2. **Waktu mulai bergerak (departure delay)**.

Tujuan optimasi adalah memperoleh konfigurasi perpindahan yang menghasilkan kepadatan dan konflik arus sekecil mungkin, tanpa membuat mahasiswa terlambat mengikuti kegiatan berikutnya.

2.2 Mengapa Permasalahan Ini Berbeda

Sebagian besar tugas Local Search (N-Queens, Traveling Salesman Problem, penjadwalan mata kuliah, knapsack, pewarnaan graf, dan sejenisnya) menilai sebuah state berdasarkan susunan atau kombinasinya saja. Pada **CampusFlow**, sebuah state tidak hanya dinilai berdasarkan susunannya, tetapi harus dijalankan melalui *simulasi waktu*. Nilai objective function dihasilkan oleh perilaku dinamis selama simulasi: berapa banyak mahasiswa yang berada pada satu koridor pada satu menit tertentu, apakah ada dua kelompok yang bertemu dari arah berlawanan, dan apakah kelompok tiba tepat waktu.

Perubahan kecil terhadap satu kelompok pun dapat berdampak besar. Menggeser keberangkatan kelompok B satu menit dapat menghilangkan penumpukan pada tiga koridor sekaligus.

3. Representasi State (Complete-State Formulation)

Satu state berisi keputusan lengkap untuk seluruh kelompok mahasiswa:

$$S = [(r_1, d_1), (r_2, d_2), \dots, (r_n, d_n)]$$

Keterangan:

- r_i adalah jalur yang digunakan kelompok ke- i ;
- d_i adalah waktu keberangkatan (delay) kelompok ke- i .

Contoh state untuk lima kelompok:

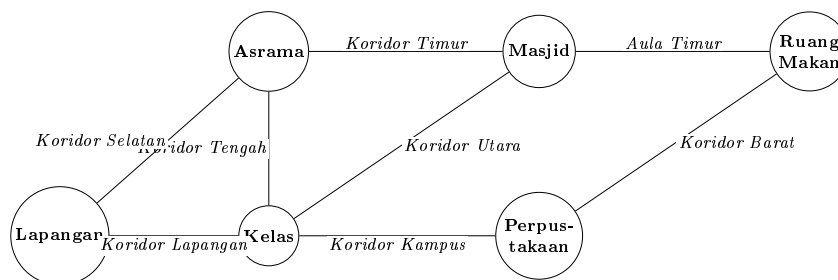
```
[ (Kelompok A, Jalur 1, Delay 0),
  (Kelompok B, Jalur 2, Delay 2),
  (Kelompok C, Jalur 3, Delay 1),
  (Kelompok D, Jalur 2, Delay 4),
  (Kelompok E, Jalur 1, Delay 3) ]
```

State tersebut sudah merupakan solusi lengkap karena seluruh kelompok telah memiliki jalur dan waktu keberangkatan. Algoritma tidak perlu mencari path dari initial state menuju goal state; kualitas solusi cukup dinilai melalui objective function.

4. Data yang Digunakan

4.1 Peta Lingkungan

Lingkungan kampus dimodelkan sebagai graf: titik lokasi adalah simpul dan koridor adalah sisi. Peta yang digunakan pada proof of concept ditunjukkan pada Gambar 1.



Gambar 1: Peta kampus POC: enam titik lokasi dan delapan koridor dua arah.

4.2 Data Kelompok

Setiap kelompok memiliki nama, jumlah mahasiswa, titik asal, titik tujuan, batas waktu tiba, daftar jalur alternatif, dan kecepatan bergerak. Dalam POC, kecepatan seluruh kelompok adalah 1 (waktu tempuh hanya ditentukan koridor).

```
{
  "nama": "Kelompok A",
  "jumlah": 30,
  "asal": "Asrama",
  "tujuan": "Masjid",
  "deadline": 8,
  "alternatif": ["E1", "E5", "E2"]
}
```

4.3 Data Jalur

Setiap koridor memiliki nama, kapasitas maksimal, waktu tempuh, dan status dua arah. Pada POC seluruh koridor bersifat dua arah.

```
{
  "nama": "Koridor Timur",
  "kapasitas": 40,
  "waktu": 2,
  "dua_arah": True
}
```

5. Aturan serta Batasan Permasalahan (Constraints)

5.1 Constraints

Sebagian constraints bersifat wajib (item 1–7): state yang melanggarnya dinyatakan tidak valid dan tidak pernah diproses lebih lanjut. Sebagian lainnya bersifat prioritas (item 8–13): boleh dilanggar, tetapi setiap pelanggaran menambah nilai cost.

1. Setiap kelompok harus memiliki tepat satu jalur.
2. Jalur harus menghubungkan titik asal dan tujuan kelompok.
3. Jalur yang sedang ditutup tidak boleh digunakan.
4. Delay harus berada dalam rentang yang diizinkan ($0 \leq d_i \leq \text{MAX_DELAY}$).
5. Setiap kelompok harus tiba sebelum batas keterlambatan maksimal (deadline + toleransi).
6. Kelompok dengan kebutuhan khusus hanya boleh menggunakan jalur aksesibel.
7. Kelompok tidak boleh berpindah melalui ruangan yang dilarang.
8. Jumlah mahasiswa dalam suatu koridor sebaiknya tidak melebihi kapasitas.
9. Dua kelompok sebaiknya tidak bertemu dari arah berlawanan.
10. Waktu tunggu kelompok sebaiknya sekecil mungkin.
11. Perbedaan delay antarkelompok sebaiknya tetap adil.
12. Jalur memutar sebaiknya tidak terlalu sering digunakan.
13. Penumpukan di pintu masuk masjid atau ruang makan harus diminimalkan.

6. Formulasi Initial State

Initial state dibuat secara acak. Untuk setiap kelompok, program:

1. Memilih salah satu jalur yang valid secara acak;
2. Memberikan delay secara acak dalam rentang yang diizinkan;
3. Memeriksa seluruh constraints;
4. Mengulang pengacakan apabila state tidak valid.

Pembangkitan initial state ditangani oleh fungsi `generate_initial_state()`. State random pertama biasanya belum efisien karena beberapa kelompok dapat menumpuk pada jalur yang sama. Contoh initial state dari POC (seed tetap 42 agar reproduibel):

```
=== INITIAL STATE (random) ===
Kelompok A : Koridor Timur (delay 0)
Kelompok B : Koridor Tengah -> Koridor Timur (delay 1)
Kelompok C : Koridor Barat (delay 1)
Kelompok D : Koridor Selatan (delay 5)

Capacity penalty : 50
Conflict penalty : 0
Lateness penalty : 0
Waiting penalty : 7
Distance penalty : 10
Fairness penalty : 3.69
Total objective cost: 531.38
```

7. Successor Function dan Neighbor Move

Neighbor diperoleh dengan melakukan perubahan kecil terhadap state saat ini. POC menyediakan empat move dasar dan satu move perturbasi.

7.1 Move 1: Mengubah Delay Satu Kelompok

Delay dinaikkan atau diturunkan satu satuan waktu:

$$d'_i = d_i \pm 1$$

Sebelum : Kelompok B -> Koridor Utara, delay 1
Sesudah : Kelompok B -> Koridor Utara, delay 2

7.2 Move 2: Mengubah Jalur Satu Kelompok

Jalur diganti dengan salah satu jalur alternatif yang masih valid.

Sebelum : Kelompok C -> Koridor Barat, delay 2
Sesudah : Kelompok C -> Koridor Kampus, delay 2

7.3 Move 3: Menukar Jalur Dua Kelompok

Kelompok A: Koridor Timur	menjadi	Kelompok A: Koridor Utara
Kelompok B: Koridor Utara	menjadi	Kelompok B: Koridor Timur

Move ini hanya diterima bila kedua kelompok tetap memenuhi constraints.

7.4 Move 4: Menukar Delay Dua Kelompok

Move ini berguna ketika salah satu kelompok lebih besar atau memiliki deadline yang lebih ketat.

7.5 Move 5: Group Perturbation

Beberapa kelompok yang memiliki titik tujuan yang sama diubah secara bersamaan. Move ini digunakan pada Simulated Annealing dan mutation pada Genetic Algorithm.

Seluruh neighbor dari `generate_neighbors(state)` dan `random_neighbor(state)` sudah lolos pemeriksaan `is_valid_state()`.

8. Objective Function

Permasalahan dirancang sebagai *minimization problem*. Semakin kecil nilai cost, semakin baik solusi.

$$Cost(S) = w_1 C_{capacity} + w_2 C_{conflict} + w_3 C_{late} + w_4 C_{waiting} + w_5 C_{distance} + w_6 C_{fairness}$$

Bobot yang digunakan pada POC:

$$Cost(S) = 10 C_{capacity} + 8 C_{conflict} + 15 C_{late} + 2 C_{waiting} + C_{distance} + 2 C_{fairness}$$

Keterlambatan diberi bobot paling besar karena dapat mengganggu jadwal kegiatan. Kelebihan kapasitas dan konflik arah juga diberi bobot tinggi karena berkaitan dengan keamanan.

8.1 Penalti Kelebihan Kapasitas

Untuk setiap ruas dan setiap waktu:

$$OverCapacity(e, t) = \max(0, Occupancy(e, t) - Capacity(e))$$

Total penalti kapasitas:

$$C_{capacity} = \sum_t \sum_e OverCapacity(e, t)^2$$

Nilai dikuadratkan agar kelebihan kapasitas yang besar mendapat hukuman lebih berat. Contoh: koridor berkapasitas 40 diisi 55 mahasiswa menghasilkan penalti $(55 - 40)^2 = 225$.

8.2 Penalti Konflik Arah

Konflik terjadi apabila dua kelompok berada pada ruas yang sama dalam waktu yang sama, tetapi bergerak dari arah berlawanan:

$$C_{conflict} = \sum_{t,e} Forward(e, t) \times Backward(e, t)$$

Semakin banyak pasangan kelompok yang bertemu dari arah berlawanan, semakin tinggi penaltinya.

8.3 Penalti Keterlambatan

$$C_{\text{late}} = \sum_i \text{Size}_i \times \max(0, \text{Arrival}_i - \text{Deadline}_i)$$

Keterlambatan dikalikan jumlah mahasiswa agar kelompok besar (misalnya 40 mahasiswa) lebih diperhatikan dibandingkan kelompok kecil (misalnya 5 mahasiswa).

8.4 Penalti Waktu Tunggu

$$C_{\text{waiting}} = \sum_i d_i$$

Komponen ini mendorong sistem meminimalkan total delay seluruh kelompok.

8.5 Penalti Jarak

$$C_{\text{distance}} = \sum_i \text{TravelTime}(\text{route}_i)$$

Komponen ini menghindari penggunaan jalur memutar yang terlalu panjang.

8.6 Penalti Keadilan

Tanpa komponen ini, algoritma mungkin terus memberikan delay besar kepada kelompok tertentu. Keadilan dihitung menggunakan varians delay:

$$C_{\text{fairness}} = \frac{1}{n} \sum_i (d_i - \bar{d})^2$$

Objective ini mendorong sistem menghasilkan pembagian waktu tunggu yang lebih proporsional.

9. Penerapan Algoritma Local Search

9.1 Empat Varian Hill-Climbing

POC mengimplementasikan empat varian Hill-Climbing dalam bentuk minimisasi:

- **Basic (Steepest-Ascent):** mengevaluasi seluruh neighbor dan memilih cost terendah yang lebih baik.
- **Sideways Move:** menerima neighbor dengan cost sama secara terbatas untuk melewati plateau.
- **Stochastic:** memilih secara acak dari kumpulan neighbor yang memperbaiki cost.
- **Random Restart:** menjalankan Basic dari beberapa initial state acak dan mengambil solusi terbaik.

Tahapan Basic pada setiap pencarian adalah:

1. Membuat initial state random.
2. Menghasilkan seluruh neighbor melalui `generate_neighbors(state)`.
3. Menghitung cost setiap neighbor.
4. Memilih neighbor dengan cost paling kecil.
5. Berpindah apabila neighbor lebih baik dari state saat ini.

6. Berhenti ketika tidak ada neighbor yang lebih baik.

Sideways Move menggunakan batas jumlah perpindahan dengan cost sama, sedangkan Stochastic memperkenalkan variasi urutan pencarian. Random Restart dijalankan enam kali karena persoalan simulasi arus memiliki banyak local optimum.

Setiap varian menyimpan urutan state dan objective function. POC menampilkan perubahan cost sebagai grafik batang teks dan mencatat kelompok yang berubah pada setiap iterasi.

9.2 Simulated Annealing

Simulated Annealing dapat menerima solusi yang lebih buruk dengan probabilitas tertentu agar dapat keluar dari local optimum:

$$P = e^{-\frac{\Delta E}{T}}$$

dengan ΔE adalah peningkatan cost dan T adalah temperature. Tahapan:

1. Membuat initial state.
2. Mengambil satu random neighbor.
3. Jika neighbor lebih baik, neighbor diterima.
4. Jika lebih buruk, neighbor diterima dengan probabilitas $e^{-\Delta E/T}$.
5. Temperature diturunkan secara geometrik ($T \leftarrow \alpha T$, $\alpha = 0,995$).
6. Proses berhenti ketika temperature minimum tercapai.

Pada awal proses, temperature tinggi sehingga penerimaan state yang lebih buruk masih diizinkan; semakin dingin, perilaku semakin menyerupai hill-climbing.

9.3 Genetic Algorithm

9.3.1 Representasi Chromosome

Satu chromosome merepresentasikan seluruh kelompok, sama seperti representasi state:

```
[ (route_A, delay_A), (route_B, delay_B),
  (route_C, delay_C), (route_D, delay_D) ]
```

9.3.2 Population, Fitness, dan Elitism

Population berisi sejumlah state random (ukuran 40 pada POC). Karena persoalan menggunakan cost minimization:

$$Fitness(S) = \frac{1}{1 + Cost(S)}$$

Beberapa chromosome terbaik (elite) dipertahankan langsung ke generasi berikutnya agar solusi terbaik tidak hilang.

9.3.3 Selection

Metode yang digunakan adalah **Tournament Selection** (ukuran turnamen 3): dipilih chromosome dengan fitness tertinggi dari beberapa kandidat acak. Tournament Selection dipilih karena lebih sederhana untuk proof of concept.

9.3.4 Crossover

One-point crossover memotong dua parent pada titik acak lalu menukar segmennya:

Parent 1 :	[A1, B1, C1 D1, E1]
Parent 2 :	[A2, B2, C2 D2, E2]
Child 1 :	[A1, B1, C1 D2, E2]
Child 2 :	[A2, B2, C2 D1, E1]

Anak yang tidak lolos constraints diganti dengan parent-nya.

9.3.5 Mutation

Mutation berupa salah satu neighbor move: mengganti jalur satu kelompok, menambah atau mengurangi delay, atau menukar delay dua kelompok. POC menggunakan probabilitas mutasi sebesar 0,4.

10. Daftar Fungsi dan Command Utama Python

Implementasi dibagi menjadi beberapa modul pada folder `src/local_search/` agar mudah dikembangkan dan diuji. Struktur modul:

File	Isi
<code>src/local_search/data.py</code>	Definisi data: koridor (EDGES), kelompok (GROUPS), konstanta (MAX_DELAY, HARD_TOLERANCE, WEIGHTS).
<code>src/local_search/simulation.py</code>	<code>simulate_flow()</code> , <code>route_time()</code> , <code>arrival_time()</code> .
<code>src/local_search/objective.py</code>	Keenam fungsi penalti dan <code>objective_function()</code> .
<code>src/local_search/state.py</code>	<code>is_valid_state()</code> , <code>generate_initial_state()</code> , <code>generate_neighbors()</code> , <code>random_neighbor()</code> .
<code>src/local_search/algorithms.py</code>	Hill-Climbing, Simulated Annealing, dan seluruh komponen Genetic Algorithm.
<code>src/local_search/view.py</code>	Seluruh fungsi pencetakan dan visualisasi teks.
<code>src/local_search/main.py</code>	Titik masuk yang menjalankan ketiga algoritma.

Fungsi-fungsi berikut diimplementasikan pada modul `src/` dan dijalankan dari direktori repository:

```
python src/local_search/main.py
```

Fungsi	Kegunaan
<code>generate_initial_state()</code>	Membuat initial state acak yang memenuhi seluruh constraints.
<code>is_valid_state(state)</code>	Memeriksa seluruh constraints untuk satu state.
<code>simulate_flow(state)</code>	Menjalankan simulasi pergerakan seluruh kelompok dan mengembalikan okupansi serta arah per koridor per menit.
<code>calculate_capacity_penalty(sim)</code>	Menghitung penalti kelebihan kapasitas koridor.
<code>calculate_conflict_penalty(sim)</code>	Menghitung penalti konflik arah (forward $\times$ backward).
<code>calculate_lateness_penalty(state)</code>	Menghitung keterlambatan tiap kelompok dikali jumlah mahasiswa.
<code>calculate_waiting_penalty(state)</code>	Menghitung total delay seluruh kelompok.
<code>calculate_distance_penalty(state)</code>	Menghitung total waktu tempuh jalur yang digunakan.
<code>calculate_fairness_penalty(state)</code>	Menghitung varians delay antarkelompok.
<code>objective_function(state)</code>	Menghitung total cost (bobot tetap yang dikonfigurasi global).
<code>generate_neighbors(state)</code>	Menghasilkan seluruh neighbor valid (Move 1-4).
<code>random_neighbor(state)</code>	Mengambil satu neighbor acak (untuk Simulated Annealing dan mutation).
<code>hill_climbing_basic(initial)</code>	Menjalankan Basic Steepest-Ascent Hill-Climbing.
<code>hill_climbing_sideways(initial)</code>	Menjalankan Sideways Move dengan batas plateau.
<code>hill_climbing_stochastic(initial)</code>	Memilih neighbor yang membaik secara stokastik.
<code>random_restart_hill_climbing(restarts)</code>	Menjalankan Basic dari beberapa initial state.
<code>simulated_annealing(initial, t0, alpha, min_temp)</code>	Menjalankan Simulated Annealing dengan pendinginan geometrik.
<code>initialize_population(size)</code>	Membuat populasi awal Genetic Algorithm.
<code>fitness(chromosome)</code>	Mengubah cost menjadi fitness $1/(1 + Cost)$.
<code>tournament_selection(population, k)</code>	Memilih parent dengan Tournament Selection.
<code>crossover(parent_1, parent_2)</code>	One-point crossover dengan perbaikan constraints.
<code>mutate(chromosome)</code>	Menerapkan satu neighbor move secara acak.
<code>genetic_algorithm(population_size, generations, mutation_rate, elite_size)</code>	Menjalankan proses evolusi dengan elitism.
<code>print_state(state)</code>	Menampilkan konfigurasi jalur dan delay seluruh kelompok.
<code>print_penalties(state)</code>	Menampilkan rincian keenam komponen penalty dan total cost.
<code>print_simulation(state)</code>	Menampilkan kondisi tiap koridor untuk setiap menit.
<code>print_summary(name, initial_cost, final_cost, iterations)</code>	Menampilkan ringkasan perbandingan hasil algoritma.

11. Contoh Input

Input POC didefinisikan pada data GROUPS dan EDGES di `src/local_search/data.py`:

```
Jumlah kelompok: 4

Kelompok A : 30 mahasiswa, Asrama -> Masjid,      deadline menit ke-8
               alternatif: [Koridor Timur], [Koridor Tengah -> Koridor Utara]
Kelompok B : 25 mahasiswa, Kelas -> Masjid,       deadline menit ke-7
               alternatif: [Koridor Utara], [Koridor Tengah -> Koridor Timur]
Kelompok C : 20 mahasiswa, Perpustakaan -> Ruang Makan, deadline menit ke-9
               alternatif: [Koridor Barat], [Koridor Kampus -> Koridor Utara -> Aula Timur]
Kelompok D : 35 mahasiswa, Lapangan -> Asrama,    deadline menit ke-10
               alternatif: [Koridor Selatan], [Koridor Lapangan -> Koridor Tengah]

Koridor (kapasitas, waktu tempuh):
Koridor Timur 40/2   Koridor Utara 30/2   Koridor Barat 25/2   Koridor Selatan 35/2
Koridor Tengah 20/2  Aula Timur 30/1     Koridor Kampus 20/3   Koridor Lapangan 25/2

Konstanta: MAX_DELAY = 6, HARD_TOLERANCE = 6, seed random = 42
```

12. Contoh Output

Seluruh output berikut adalah hasil nyata dari POC.

12.1 Initial State dan Proses Pencarian (Hill-Climbing)

```

=== INITIAL STATE (random) ===
Kelompok A : Koridor Timur (delay 0)
Kelompok B : Koridor Tengah -> Koridor Timur (delay 1)
Kelompok C : Koridor Barat (delay 1)
Kelompok D : Koridor Selatan (delay 5)

Capacity penalty : 50
Conflict penalty : 0
Lateness penalty : 0
Waiting penalty : 7
Distance penalty : 10
Fairness penalty : 3.69
Total objective cost: 531.38

Restart 1: 525.38 -> 23.38 -> 19.00 -> 15.38 -> 12.50 -> 10.38 -> 8.00
Restart 2: 8796.50 -> 2294.50 -> 40.50 -> 37.38 -> 35.00 -> 32.38 -> 29.50
          -> 27.38 -> 25.00 -> 22.38 -> 20.50 -> 18.38 -> 16.00 -> 14.38
          -> 12.50 -> 10.38 -> 8.00
...
Best cost setelah 6 restart : 8.00

```

12.2 Proses Pencarian (Simulated Annealing)

```

Iterasi 0..9      : 537.00 35.00 35.00 39.00 40.38 36.38 36.38 36.38 36.38 36.00
Tengah simulasi   : 21.38 25.38 25.38 25.38 25.38
Akhir simulasi    : 8.00 8.00 8.00 8.00 8.00
Total iterasi     : 1094

```

12.3 Proses Pencarian (Genetic Algorithm)

```

Best fitness per generasi (setiap 10):
Generasi 0      : 36.50
Generasi 10     : 8.00
Generasi 20     : 8.00
Generasi 30     : 8.00
Generasi 40     : 8.00
Generasi 50     : 8.00
Generasi 59     : 8.00

```

12.4 Final State

```

=== FINAL STATE (solusi terbaik) ===
Kelompok A : Koridor Timur (delay 0)
Kelompok B : Koridor Utara (delay 0)
Kelompok C : Koridor Barat (delay 0)
Kelompok D : Koridor Selatan (delay 0)

```

```
Capacity penalty : 0
Conflict penalty : 0
Lateness penalty : 0
Waiting penalty : 0
Distance penalty : 8
Fairness penalty : 0.00
Total objective cost: 8.00
```

Semua kelompok berangkat tanpa delay pada jalur langsung; tidak ada koridor yang kelebihan kapasitas, tidak ada konflik arah, dan tidak ada keterlambatan. Cost akhir hanya berasal dari penalti jarak $8 = 2 + 2 + 2 + 2$.

12.5 Visualisasi Teks Simulasi

Perbandingan kondisi koridor pada menit yang sama, antara initial state dan final state:

```
INITIAL STATE, MENIT 1:
Koridor Timur : [#####.] 30/40
Koridor Barat : [#####.] 20/25
Koridor Tengah : [#####] 25/20 <-- melebihi kapasitas 20

FINAL STATE, MENIT 1:
Koridor Timur : [#####.] 30/40
Koridor Utara : [#####.] 25/30
Koridor Barat : [#####.] 20/25
Koridor Selatan : [#####] 35/35
```

Pada initial state, Koridor Tengah menampung 25 mahasiswa padahal kapasitasnya 20 (penalti $2 \times 5^2 = 50$). Pada final state, seluruh kelompok berpindah serentak pada jalur langsung sehingga tidak ada koridor yang melebihi kapasitas dan seluruh kelompok tiba tepat waktu.

12.6 Ringkasan Hasil

```
=== RINGKASAN HASIL ===
Initial cost : 531.38

Algorithm      : Random-Restart Hill Climbing
Final cost     : 8.00
Improvement    : 98.49%
Total iteration : 7

Algorithm      : Simulated Annealing
Final cost     : 8.00
Improvement    : 98.49%
Total iteration : 1094

Algorithm      : Genetic Algorithm
Final cost     : 8.00
Improvement    : 98.49%
Total iteration : 60
```

Ketiga algoritma berhasil mencapai solusi optimal yang sama (cost 8,00) pada konfigurasi POC ini, dengan jumlah iterasi yang sangat berbeda. Hill-Climbing merupakan algoritma tercepat (7 langkah), diikuti Genetic Algorithm (60 generasi) dan Simulated Annealing (1094 iterasi).

13. Batas Proof of Concept, Pengujian, dan Deliverables

13.1 Batas Proof of Concept

Proof of concept yang disertakan pada repository mengimplementasikan fitur utama dan bonus Local Search, serta menjadi bukti bahwa rancangan dapat dijalankan. POC bukan implementasi akhir seluruh fitur bonus lainnya.

Fitur	Status POC	Kriteria Selesai
Initial state acak valid	Sudah	<code>generate_initial_state()</code> selalu menghasilkan state yang lolos constraints.
Simulasi arus per menit	Sudah	<code>simulate_flow()</code> menghitung okupansi dan arah tiap koridor per menit.
Objective function enam komponen	Sudah	Seluruh penalti terhitung dan terperinci pada output.
Hill-Climbing	Sudah	Basic, Sideways Move, Stochastic, Random Restart, dan pencatatan history.
Simulated Annealing	Sudah	Penerimaan state lebih buruk berbasis probabilitas.
Genetic Algorithm	Sudah	Populasi, tournament selection, one-point crossover, mutation, dan elitism.
Visualisasi teks	Sudah	State awal, state akhir, history cost, perubahan state per iterasi, dan simulasi per menit ditampilkan.
Perbandingan kuantitatif	Sudah	Ringkasan cost awal, cost akhir, improvement, dan jumlah iterasi.
Peta kompleks (banyak lokasi)	Belum	Dapat ditambahkan pada implementasi akhir.
Koridor ditutup dinamis	Belum	constraint jalur tertutup baru diaktifkan sebagian.
Antarmuka grafis	Belum	Implementasi akhir tetap berbasis terminal.

13.2 Pengujian

- Random generator menggunakan `random.seed(42)` sehingga seluruh hasil dapat direproduksi.
- Setiap state dan neighbor diperiksa dengan `is_valid_state()` sebelum diproses.
- Hasil tiga algoritma dibandingkan terhadap cost akhir yang sama (optimal) untuk memverifikasi konsistensi objective function.

13.3 Deliverables

- Dokumen spesifikasi ini (sumber `.tex` dan hasil `.pdf`).
- `src/local_search/main.py` dan modul pendukung `src/`: proof of concept Python.
- `src/local_search/poc_output.txt`: output lengkap hasil menjalankan POC.

14. Penutup

CampusFlow merupakan persoalan local search yang lengkap: complete-state formulation yang jelas, neighbor function berbasis empat move sederhana, objective function multikriteria enam komponen, serta peran yang relevan untuk Random-Restart Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Proof of concept membuktikan ketiga algoritma berhasil menurunkan

cost dari 531,38 menjadi 8,00 (perbaikan 98,49%) pada konfigurasi uji. Rancangan ini dapat dikembangkan lebih lanjut menjadi simulasi keamanan, evakuasi, atau manajemen fasilitas kampus tanpa mengubah kontrak fungsi utama.

A. Lampiran: Kode Sumber Proof of Concept

Kode sumber lengkap POC dibagi menjadi tujuh modul pada folder `src/local_search/`. Untuk menjalankan seluruh simulasi: `python src/local_search/main.py`. Berikut isi setiap modul.

A.1 `src/local_search/data.py`: Data Lingkungan

```

EDGES = {
    "E1": {"nama": "Koridor Timur", "a": "Asrama", "b": "Masjid", "kapasitas": 40, "waktu": 2},
    "E2": {"nama": "Koridor Utara", "a": "Kelas", "b": "Masjid", "kapasitas": 30, "waktu": 2},
    "E3": {"nama": "Koridor Barat", "a": "Perpustakaan", "b": "Ruang Makan", "kapasitas": 25, "waktu": 2},
    "E4": {"nama": "Koridor Selatan", "a": "Lapangan", "b": "Asrama", "kapasitas": 35, "waktu": 2},
    "E5": {"nama": "Koridor Tengah", "a": "Asrama", "b": "Kelas", "kapasitas": 20, "waktu": 2},
    "E6": {"nama": "Aula Timur", "a": "Masjid", "b": "Ruang Makan", "kapasitas": 30, "waktu": 1},
    "E7": {"nama": "Koridor Kampus", "a": "Perpustakaan", "b": "Kelas", "kapasitas": 20, "waktu": 3},
    "E8": {"nama": "Koridor Lapangan", "a": "Lapangan", "b": "Kelas", "kapasitas": 25, "waktu": 2},
}

GROUPS = [
    {"nama": "Kelompok A", "jumlah": 30, "asal": "Asrama", "tujuan": "Masjid", "deadline": 8,
     "alternatif": [["E1"], ["E5", "E2"]]},
    {"nama": "Kelompok B", "jumlah": 25, "asal": "Kelas", "tujuan": "Masjid", "deadline": 7,
     "alternatif": [["E2"], ["E5", "E1"]]},
    {"nama": "Kelompok C", "jumlah": 20, "asal": "Perpustakaan", "tujuan": "Ruang Makan", "deadline": 9,
     "alternatif": [["E3"], ["E7", "E2", "E6"]]},
    {"nama": "Kelompok D", "jumlah": 35, "asal": "Lapangan", "tujuan": "Asrama", "deadline": 10,
     "alternatif": [["E4"], ["E8", "E5"]]},
]

MAX_DELAY = 6
HARD_TOLERANCE = 6
WEIGHTS = {"capacity": 10, "conflict": 8, "late": 15, "waiting": 2, "distance": 1, "fairness": 2}

```

A.2 `src/local_search/simulation.py`: Simulasi Arus

```

from data import EDGES, GROUPS

def route_time(route):
    return sum(EDGES[e]["waktu"] for e in route)

def arrival_time(route, delay):
    return delay + route_time(route)

```



```

def simulate_flow(state):
    sim = {}
    for idx, (route, delay) in enumerate(state):
        t = delay
        node = GROUPS[idx]["asal"]
        for e in route:
            info = EDGES[e]
            if node == info["a"]:
                fwd = True
                node = info["b"]
            else:
                fwd = False
                node = info["a"]
            for minute in range(t, t + info["waktu"]):
                entry = sim.setdefault((e, minute), {"occ": 0, "fwd": 0, "bwd": 0})
                entry["occ"] += GROUPS[idx]["jumlah"]
                if fwd:
                    entry["fwd"] += 1
                else:
                    entry["bwd"] += 1
            t += info["waktu"]
    return sim

```

A.3 src/local_search/objective.py: Objective Function

```

from data import EDGES, GROUPS, WEIGHTS
from simulation import simulate_flow, route_time, arrival_time

def calculate_capacity_penalty(sim):
    total = 0
    for (e, minute), entry in sim.items():
        over = entry["occ"] - EDGES[e]["kapasitas"]
        if over > 0:
            total += over * over
    return total

def calculate_conflict_penalty(sim):
    return sum(entry["fwd"] * entry["bwd"] for entry in sim.values())

def calculate_lateness_penalty(state):
    total = 0
    for group, (route, delay) in zip(GROUPS, state):
        late = arrival_time(route, delay) - group["deadline"]
        if late > 0:
            total += group["jumlah"] * late
    return total

def calculate_waiting_penalty(state):
    return sum(delay for _, delay in state)

def calculate_distance_penalty(state):
    return sum(route_time(route) for route, _ in state)

def calculate_fairness_penalty(state):

```

```

delays = [delay for _, delay in state]
if not delays:
    return 0
mean = sum(delays) / len(delays)
return sum((d - mean) ** 2 for d in delays) / len(delays)

def objective_function(state):
    sim = simulate_flow(state)
    return (WEIGHTS["capacity"] * calculate_capacity_penalty(sim)
            + WEIGHTS["conflict"] * calculate_conflict_penalty(sim)
            + WEIGHTS["late"] * calculate_lateness_penalty(state)
            + WEIGHTS["waiting"] * calculate_waiting_penalty(state)
            + WEIGHTS["distance"] * calculate_distance_penalty(state)
            + WEIGHTS["fairness"] * calculate_fairness_penalty(state))

```

A.4 src/local_search/state.py: State, Initial State, dan Neighbor

```

import random

from data import GROUPS, MAX_DELAY, HARD_TOLERANCE
from simulation import arrival_time

def is_valid_state(state):
    for group, (route, delay) in zip(GROUPS, state):
        if route not in group["alternatif"]:
            return False
        if not (0 <= delay <= MAX_DELAY):
            return False
        if arrival_time(route, delay) > group["deadline"] + HARD_TOLERANCE:
            return False
    return True

def generate_initial_state():
    while True:
        state = tuple((random.choice(g["alternatif"]), random.randint(0, MAX_DELAY)) for g
            in GROUPS)
        if is_valid_state(state):
            return state

def generate_neighbors(state):
    neighbors = []
    n = len(state)
    for i in range(n):
        route, delay = state[i]
        for delta in (-1, 1):
            nd = delay + delta
            if 0 <= nd <= MAX_DELAY:
                nbr = list(state)
                nbr[i] = (route, nd)
                nbr = tuple(nbr)
                if is_valid_state(nbr):
                    neighbors.append(nbr)
    for i in range(n):
        route, delay = state[i]
        for alt in GROUPS[i]["alternatif"]:
            if alt != route:

```

```

        nbr = list(state)
        nbr[i] = (alt, delay)
        nbr = tuple(nbr)
        if is_valid_state(nbr):
            neighbors.append(nbr)
    for i in range(n):
        for j in range(i + 1, n):
            nbr = list(state)
            nbr[i] = (state[j][0], state[i][1])
            nbr[j] = (state[i][0], state[j][1])
            nbr = tuple(nbr)
            if is_valid_state(nbr):
                neighbors.append(nbr)
    for i in range(n):
        for j in range(i + 1, n):
            nbr = list(state)
            nbr[i] = (state[i][0], state[j][1])
            nbr[j] = (state[j][0], state[i][1])
            nbr = tuple(nbr)
            if is_valid_state(nbr):
                neighbors.append(nbr)
    return neighbors

def random_neighbor(state):
    n = len(state)
    if n < 2:
        return state
    for _ in range(60):
        nbr = list(state)
        move = random.randint(0, 3)
        if move == 0:
            i = random.randrange(n)
            route, delay = state[i]
            delta = random.choice((-1, 1))
            nd = delay + delta
            if 0 <= nd <= MAX_DELAY:
                nbr[i] = (route, nd)
        elif move == 1:
            i = random.randrange(n)
            route, delay = state[i]
            alts = [a for a in GROUPS[i]["alternatif"] if a != route]
            if alts:
                nbr[i] = (random.choice(alts), delay)
        elif move == 2:
            i, j = random.sample(range(n), 2)
            nbr[i] = (state[j][0], state[i][1])
            nbr[j] = (state[i][0], state[j][1])
        else:
            i, j = random.sample(range(n), 2)
            nbr[i] = (state[i][0], state[j][1])
            nbr[j] = (state[j][0], state[i][1])
        nbr = tuple(nbr)
        if nbr != state and is_valid_state(nbr):
            return nbr
    return state

```

A.5 src/local_search/algorithms.py: Algoritma Local Search

```

import math
import random

from objective import objective_function
from state import generate_initial_state, generate_neighbors, is_valid_state,
    random_neighbor

def _search_result(current, costs, states, return_states):
    if return_states:
        return current, costs, states
    return current, costs

def hill_climbing_basic(initial, max_iterations=1000, return_states=False):
    current = initial
    current_cost = objective_function(current)
    costs = [current_cost]
    states = [current]

    for _ in range(max_iterations):
        neighbors = generate_neighbors(current)
        if not neighbors:
            break
        best = min(neighbors, key=objective_function)
        best_cost = objective_function(best)
        if best_cost >= current_cost:
            break
        current = best
        current_cost = best_cost
        costs.append(current_cost)
        states.append(current)

    return _search_result(current, costs, states, return_states)

def hill_climbing_sideways(initial, max_iterations=1000, max_sideways=10,
    return_states=False):
    current = initial
    current_cost = objective_function(current)
    costs = [current_cost]
    states = [current]
    sideways_used = 0

    for _ in range(max_iterations):
        neighbors = generate_neighbors(current)
        if not neighbors:
            break
        best_cost = min(objective_function(nbr) for nbr in neighbors)
        candidates = [
            nbr for nbr in neighbors
            if objective_function(nbr) == best_cost
        ]
        if best_cost > current_cost:
            break
        if best_cost == current_cost:
            if sideways_used >= max_sideways:
                break
            sideways_used += 1
        else:

```

```

        sideways_used = 0
        current = random.choice(candidates)
        current_cost = best_cost
        costs.append(current_cost)
        states.append(current)

    return _search_result(current, costs, states, return_states)

def hill_climbing_stochastic(initial, max_iterations=1000, return_states=False):
    current = initial
    current_cost = objective_function(current)
    costs = [current_cost]
    states = [current]

    for _ in range(max_iterations):
        neighbors = generate_neighbors(current)
        improving = [
            nbr for nbr in neighbors
            if objective_function(nbr) < current_cost
        ]
        if not improving:
            break
        current = random.choice(improving)
        current_cost = objective_function(current)
        costs.append(current_cost)
        states.append(current)

    return _search_result(current, costs, states, return_states)

def hill_climbing(initial, max_iterations=1000, return_states=False):
    return hill_climbing_basic(initial, max_iterations, return_states)

def random_restart_hill_climbing(restarts=6, max_iterations=1000,
                                return_states=False):
    if restarts <= 0:
        raise ValueError("restarts harus lebih besar dari 0")
    best_state = None
    best_cost = float("inf")
    all_histories = []
    all_states = []
    for _ in range(restarts):
        start = generate_initial_state()
        final, history, states = hill_climbing_basic(
            start, max_iterations=max_iterations, return_states=True
        )
        all_histories.append(history)
        all_states.append(states)
        if objective_function(final) < best_cost:
            best_cost = objective_function(final)
            best_state = final
    if return_states:
        return best_state, all_histories, all_states
    return best_state, all_histories

def simulated_annealing(initial, t0=120.0, alpha=0.995, min_temp=0.5):
    current = initial
    temp = t0
    history = []

```

```

while temp > min_temp:
    nbr = random_neighbor(current)
    delta = objective_function(nbr) - objective_function(current)
    if delta < 0 or random.random() < math.exp(-delta / temp):
        current = nbr
    history.append(objective_function(current))
    temp *= alpha
return current, history

def initialize_population(size=40):
    return [generate_initial_state() for _ in range(size)]

def fitness(chromosome):
    return 1.0 / (1.0 + objective_function(chromosome))

def tournament_selection(population, k=3):
    best = None
    for _ in range(k):
        candidate = random.choice(population)
        if best is None or fitness(candidate) > fitness(best):
            best = candidate
    return best

def crossover(parent_1, parent_2):
    cut = random.randint(1, len(parent_1) - 1)
    child_1 = parent_1[:cut] + parent_2[cut:]
    child_2 = parent_2[:cut] + parent_1[cut:]
    if not is_valid_state(child_1):
        child_1 = parent_1
    if not is_valid_state(child_2):
        child_2 = parent_2
    return child_1, child_2

def mutate(chromosome):
    return random_neighbor(chromosome)

def genetic_algorithm(population_size=40, generations=60, mutation_rate=0.4, elite_size=4):
    population = initialize_population(population_size)
    history = []
    for _ in range(generations):
        population.sort(key=objective_function)
        history.append(objective_function(population[0]))
        new_population = population[:elite_size]
        while len(new_population) < population_size:
            p1 = tournament_selection(population)
            p2 = tournament_selection(population)
            c1, c2 = crossover(p1, p2)
            child = c1 if fitness(c1) >= fitness(c2) else c2
            if random.random() < mutation_rate:
                child = mutate(child)
            new_population.append(child)
        population = new_population
    population.sort(key=objective_function)
    return population[0], history

```

A.6 src/local_search/view.py: Visualisasi Teks

```

from data import EDGES, GROUPS
from objective import (calculate_capacity_penalty, calculate_conflict_penalty,
                      calculate_distance_penalty, calculate_fairness_penalty,
                      calculate_lateness_penalty, calculate_waiting_penalty,
                      objective_function)
from simulation import simulate_flow

def route_name(route):
    return " -> ".join(EDGES[e]["nama"] for e in route)

def print_state(state):
    for group, (route, delay) in zip(GROUPS, state):
        print(f"{group['nama']:<11} : {route_name(route)} (delay {delay})")

def print_penalties(state):
    sim = simulate_flow(state)
    print(f"Capacity penalty : {calculate_capacity_penalty(sim)}")
    print(f"Conflict penalty : {calculate_conflict_penalty(sim)}")
    print(f"Lateness penalty : {calculate_lateness_penalty(state)}")
    print(f"Waiting penalty : {calculate_waiting_penalty(state)}")
    print(f"Distance penalty : {calculate_distance_penalty(state)}")
    print(f"Fairness penalty : {calculate_fairness_penalty(state):.2f}")
    print(f"Total objective cost: {objective_function(state):.2f}")

def print_simulation(state):
    from simulation import arrival_time
    horizon = max(arrival_time(r, d) for (r, d) in state) + 1
    sim = simulate_flow(state)
    for minute in range(horizon):
        print(f"MENIT {minute}")
        for e in EDGES:
            entry = sim.get((e, minute))
            occ = entry["occ"] if entry else 0
            cap = EDGES[e]["kapasitas"]
            filled = min(10, int(round(occ / cap * 10))) if cap else 0
            bar = "#" * filled + "." * (10 - filled)
            print(f"{EDGES[e]['nama']:<14} : [{bar}] {occ}/{cap}")

def print_summary(name, initial_cost, final_cost, iterations):
    improvement = 0 if initial_cost == 0 else (initial_cost - final_cost) / initial_cost * 100
    print(f"Algorithm      : {name}")
    print(f"Initial cost    : {initial_cost:.2f}")
    print(f"Final cost      : {final_cost:.2f}")
    print(f"Improvement     : {improvement:.2f}%")
    print(f"Total iteration : {iterations}")

def print_search_visualization(name, state_history, cost_history, max_steps=25):
    print(f"\n=== VISUALISASI PENCARIAN: {name} ===")
    if not cost_history:
        print("Tidak ada iterasi.")
        return
    shown = min(len(cost_history), max_steps)
    ceiling = max(cost_history)

```

```

floor = min(cost_history)
span = max(ceiling - floor, 1e-12)
for index in range(shown):
    cost = cost_history[index]
    width = int(round((cost - floor) / span * 36))
    changed = "awal" if index == 0 else _changed_groups(
        state_history[index - 1], state_history[index]
    )
    print(f"Iterasi {index:>3} | {cost:>8.2f} | {'#' * width:<36} | {changed}")
if len(cost_history) > shown:
    print(f"... {len(cost_history) - shown} iterasi berikutnya disembunyikan")

def _changed_groups(previous, current):
    changed = [
        str(index + 1)
        for index, (before, after) in enumerate(zip(previous, current))
        if before != after
    ]
    return "kelompok " + ", ".join(changed) if changed else "tidak berubah"

```

A.7 src/local_search/main.py: Titik Masuk Program

```

import random

from algorithms import (genetic_algorithm, hill_climbing_basic,
                        hill_climbing_sideways, hill_climbing_stochastic,
                        random_restart_hill_climbing, simulated_annealing)
from objective import objective_function
from state import generate_initial_state
from view import (print_penalties, print_search_visualization, print_simulation,
                  print_state, print_summary)

random.seed(42)

def main():
    print("=" * 64)
    print("CAMPUSFLOW - PROOF OF CONCEPT")
    print("Optimasi Pergerakan Kelompok Mahasiswa Menggunakan Local Search")
    print("=" * 64)

    initial = generate_initial_state()
    print("\n=== INITIAL STATE (random) ===")
    print_state(initial)
    print()
    print_penalties(initial)
    print("\n=== VISUALISASI SIMULASI INITIAL STATE ===")
    print_simulation(initial)

    print("\n" + "=" * 64)
    print("1. VARIAN HILL-CLIMBING")
    print("=" * 64)
    basic_state, basic_history, basic_states = hill_climbing_basic(
        initial, return_states=True
    )
    sideways_state, sideways_history, sideways_states = hill_climbing_sideways(
        initial, return_states=True
    )
    stochastic_state, stochastic_history, stochastic_states = hill_climbing_stochastic(

```



```

        initial, return_states=True
    )
    hc_state, hc_histories, hc_state_histories = random_restart_hill_climbing(
        restarts=6, return_states=True
    )
    hill_variants = [
        ("Basic (Steepest-Ascent)", basic_state, basic_history, basic_states),
        ("Sideways Move", sideways_state, sideways_history, sideways_states),
        ("Stochastic", stochastic_state, stochastic_history, stochastic_states),
    ]
    for name, state, history, states in hill_variants:
        print(f"\n{name}")
        print("Cost: " + " -> ".join(f"{cost:.2f}" for cost in history))
        print_search_visualization(name, states, history)
    print("\nRandom Restart")
    for i, h in enumerate(hc_histories):
        trace = " -> ".join(f"{c:.2f}" for c in h)
        print(f"Restart {i + 1}: {trace}")
    print(f"\nBest cost setelah 6 restart : {objective_function(hc_state):.2f}")
    best_restart_index = min(
        range(len(hc_histories)),
        key=lambda index: hc_histories[index][-1],
    )
    print_search_visualization(
        "Random Restart",
        hc_state_histories[best_restart_index],
        hc_histories[best_restart_index],
    )
    print("\n=== FINAL STATE (Hill Climbing) ===")
    print_state(hc_state)
    print()
    print_penalties(hc_state)

    print("\n" + "=" * 64)
    print("2. SIMULATED ANNEALING")
    print("=" * 64)
    sa_state, sa_history = simulated_annealing(initial)
    print("Iterasi 0..9      : " + " ".join(f"{c:.2f}" for c in sa_history[:10]))
    print("Tengah simulasi   : " + " ".join(f"{c:.2f}" for c in sa_history[len(sa_history)
        // 2:len(sa_history) // 2 + 5]))
    print("Akhir simulasi    : " + " ".join(f"{c:.2f}" for c in sa_history[-5:]))
    print(f"Total iterasi      : {len(sa_history)}")
    print("\n=== FINAL STATE (Simulated Annealing) ===")
    print_state(sa_state)
    print()
    print_penalties(sa_state)

    print("\n" + "=" * 64)
    print("3. GENETIC ALGORITHM")
    print("=" * 64)
    ga_state, ga_history = genetic_algorithm()
    print("Best fitness per generasi (setiap 10):")
    for i in range(0, len(ga_history), 10):
        print(f"Generasi {i:<4} : {ga_history[i]:.2f}")
    print(f"Generasi {len(ga_history) - 1:<4} : {ga_history[-1]:.2f}")
    print("\n=== FINAL STATE (Genetic Algorithm) ===")
    print_state(ga_state)
    print()
    print_penalties(ga_state)

    candidates = [("Basic Hill Climbing", basic_state),
        ("Sideways Move Hill Climbing", sideways_state),

```

```

        ("Stochastic Hill Climbing", stochastic_state),
        ("Random-Restart Hill Climbing", hc_state),
        ("Simulated Annealing", sa_state),
        ("Genetic Algorithm", ga_state)]
best_name, best_state = min(candidates, key=lambda t: objective_function(t[1]))

print("\n" + "=" * 64)
print("4. RINGKASAN PERBANDINGAN ALGORITMA")
print("=" * 64)
for name, state in candidates:
    print(f"{name:<30} : cost akhir {objective_function(state):.2f}")
print(f"\nSolusi terbaik: {best_name}")

print("\n=== FINAL STATE (solusi terbaik) ===")
print_state(best_state)
print()
print_penalties(best_state)
print("\n=== VISUALISASI SIMULASI FINAL STATE ===")
print_simulation(best_state)

print("\n=== RINGKASAN HASIL ===")
hc_iters = len(min(hc_histories, key=len))
summary_data = [
    ("Basic Hill Climbing", basic_state, len(basic_history)),
    ("Sideways Move Hill Climbing", sideways_state, len(sideways_history)),
    ("Stochastic Hill Climbing", stochastic_state, len(stochastic_history)),
    ("Random-Restart Hill Climbing", hc_state, hc_iters),
    ("Simulated Annealing", sa_state, len(sa_history)),
    ("Genetic Algorithm", ga_state, len(ga_history)),
]
for name, state, iters in summary_data:
    print_summary(name, objective_function(initial), objective_function(state), iters)

if __name__ == "__main__":
    main()

```